

Authorize.Net Integration Tutorial

Wolastoq Bingo — Saint Mary's Entertainment Centre

Prepared for Sir Hilbert

May 7, 2026

Contents

Authorize.Net Integration — Developer Tutorial	2
How to use this tutorial	2
Stage 0 — Prerequisites	2
0.1 Credentials you need from the project owner	2
0.2 Local development setup checklist	2
0.3 Read these files before coding	3
0.4 The big picture — what changes	3
Stage 1 — Sandbox account and a “Hello World” test charge	3
1.1 Sandbox dashboard tour	4
1.2 Smoke test from the command line	4
Stage 2 — Database migration	4
2.1 The migration	5
2.2 Run the migration	5
Stage 3 — Environment variables	6
Stage 4 — Install the SDK	6
Stage 5 — The payments service	6
Stage 6 — Refactor the booking endpoint	11
6.1 Add helper functions	11
6.2 Refactor POST /api/bookings → POST /api/bookings/initiate	12
6.3 Add the return / cancel / status / webhook endpoints	13
6.4 What to do with the OLD POST /api/bookings	15
Stage 7 — Frontend changes	15
7.1 Update the booking submit	15
7.2 Processing page	16
7.3 Wire up the routes	17
7.4 Build the client	17
Stage 8 — Webhook setup	17
8.1 Expose your local server to the internet	17
8.2 Register the webhook	17
8.3 Test the webhook	18
Stage 9 — Admin refund/void UI	18
9.1 Backend endpoints	18
9.2 Admin UI	19
Stage 10 — End-to-end testing	19
Stage 11 — Production cutover	20

Authorize.Net Integration — Developer Tutorial

Project: Wolastoq Bingo (Saint Mary’s Entertainment Centre Booking System) **Stack:** Node.js + Express 4 + Socket.io 4 + sql.js (SQLite) / React 18 + Vite **Integration style:** Accept Hosted (Authorize.Net hosts the card form — lowest PCI burden) **Audience:** The developer integrating payments **Estimated time:** 3-5 working days including testing

How to use this tutorial

Work through the stages in order. Each stage has:

- **Goal** — what the stage accomplishes
- **Files touched** — exactly which files you’ll edit
- **Code** — copy-paste blocks
- **What to verify** — how you confirm the stage is working before moving on

Where you see [Screenshot] INSERT SCREENSHOT HERE — that’s a spot for the project owner to paste in a screenshot from their own Authorize.Net dashboard. The text underneath describes what should be on that screen so you know you’re in the right place.

Stage 0 — Prerequisites

0.1 Credentials you need from the project owner

The existing live Wolastoq Bingo system already uses Authorize.Net, so the merchant account and gateway are already provisioned. Ask the project owner for:

Item	What it looks like	Used for
Sandbox API Login ID	string, ~12 chars	All API calls during dev
Sandbox Transaction Key	16-char alphanumeric	Server-side authentication
Sandbox Signature Key	128-char hex string	Verifying webhook authenticity (HMAC-SHA512)
Production API Login ID	string	Only at go-live, keep this in a separate vault
Production Transaction Key	string	Only at go-live
Production Signature Key	string	Only at go-live

If sandbox credentials don’t exist yet, create them at <https://developer.authorize.net/hello-world/sandbox> (free, no credit card required). Each developer can have their own sandbox account.

[INSERT SCREENSHOT HERE] — Authorize.Net Merchant Interface → **Account** → **API Credentials & Keys** page. You’ll see your API Login ID at the top and a “New Transaction Key” button below.

0.2 Local development setup checklist

- ☐ git clone the repo
- ☐ npm install at root, in /server, and in /client

- ❑ Copy server/.env.example to server/.env (or get the existing .env from the project owner)
- ❑ Confirm you can run the existing system locally:
 - npm run dev (starts both server on port 3001 and client on port 3000)
 - Visit http://localhost:3000 and create a test booking — should work without payment today
- ❑ Confirm the database file server/bingo.db exists and has data (run npm run seed if not)

0.3 Read these files before coding

You'll be touching them — read first to get oriented:

File	Purpose
server/src/index.js	Main Express app — all routes are here. ~1500 lines.
server/src/database.js	sql.js wrapper. Note the debounced save behavior.
server/src/migrate.js	Schema migrations. New columns go here.
server/src/services/email.js	Sends booking confirmations. We'll call this from the new flow.
server/src/services/holds.js	Seat hold timeouts. We'll need to extend hold time during payment.
client/src/api.js	Frontend API helper. Booking submit lives here.

0.4 The big picture — what changes

Today's flow:

Customer clicks "Confirm"

- POST /api/bookings
- seats flipped to 'sold' instantly
- booking inserted as payment_status='paid' ← no money was taken!
- confirmation email sent

New flow (what you're building):

Customer clicks "Confirm"

- POST /api/bookings/initiate
- booking inserted as payment_status='pending'
- seats stay 'held'
- server gets a hosted-payment-page token from Authorize.Net
- returns { returnUrl, token, bookingId } to client
- Client auto-submits a form to Authorize.Net's hosted page

Customer pays on Authorize.Net's page

- Authorize.Net redirects browser back to /payment/return?bookingId=XYZ
- Authorize.Net calls our webhook → POST /api/webhooks/authorize-net
- Whichever fires first wins, the other is a no-op (idempotent)
 - On success: flip booking to 'paid', flip seats to 'sold', send email
 - On failure: mark 'failed', release seats back to 'held'

The key principle: **never mark anything paid until Authorize.Net confirms it.**

Stage 1 — Sandbox account and a “Hello World” test charge

Goal: Make sure your sandbox credentials work before writing any integration code.

Files touched: none (you'll just hit the API from a curl/Postman test).

1.1 Sandbox dashboard tour

Log in at <https://sandbox.authorize.net> with the sandbox credentials.

[INSERT SCREENSHOT HERE] — Sandbox dashboard home page. You'll see a left nav with "Tools," "Reports," "Account," etc. The header shows "Test Mode" in red text — that's how you know you're in the sandbox.

Navigate to **Account → API Credentials & Keys**. Confirm you have: - API Login ID (visible) - A Transaction Key (you may need to generate one — pick "New Transaction Key", click Submit, save the value somewhere — Authorize.Net shows it ONCE) - A Signature Key (same — generate, save once)

[INSERT SCREENSHOT HERE] — The "API Credentials & Keys" page showing the API Login ID field, "New Transaction Key" radio button, and "Signature Key" section.

1.2 Smoke test from the command line

This proves the credentials work without writing any app code:

```
# Replace the API_LOGIN_ID and TRANSACTION_KEY values
curl -X POST https://apitest.authorize.net/xml/v1/request.api \
-H "Content-Type: application/json" \
-d '{
  "createTransactionRequest": {
    "merchantAuthentication": {
      "name": "YOUR_API_LOGIN_ID",
      "transactionKey": "YOUR_TRANSACTION_KEY"
    },
    "transactionRequest": {
      "transactionType": "authCaptureTransaction",
      "amount": "1.00",
      "payment": {
        "creditCard": {
          "cardNumber": "4111111111111111",
          "expirationDate": "2030-12",
          "cardCode": "123"
        }
      }
    }
  }
}'
```

What to verify: - Response includes "resultCode": "Ok" and "transactionResponse"."responseCode": "1" (= approved) - The transaction shows up in the sandbox dashboard under **Transactions → Unsettled Transactions**

If you get an error like "E00007 - User authentication failed", the credentials are wrong. Stop here and re-check before moving on.

[INSERT SCREENSHOT HERE] — Sandbox dashboard, **Transactions → Unsettled Transactions**, showing the \$1.00 test transaction you just made.

Stage 2 — Database migration

Goal: Add columns to bookings so we can track the Authorize.Net transaction lifecycle. Add a payment_events table for audit trail.

Files touched: server/src/migrate.js

2.1 The migration

Open server/src/migrate.js. You'll see existing migrations following a pattern. Add a new migration block at the end:

```
// server/src/migrate.js – append to the existing migrations function

// --- Migration: Authorize.Net payment columns ---
function migrationAuthorizeNet() {
  // sql.js doesn't support "ALTER TABLE ... IF NOT EXISTS" so we check first
  const cols = all("PRAGMA table_info(bookings)").map(c => c.name);

  const addCol = (name, type) => {
    if (!cols.includes(name)) {
      exec(`ALTER TABLE bookings ADD COLUMN ${name} ${type}`);
    }
  };

  addCol('payment_provider', "TEXT DEFAULT 'authorize_net'");
  addCol('transaction_id', 'TEXT');
  addCol('auth_code', 'TEXT');
  addCol('hosted_token', 'TEXT');
  addCol('payment_attempted_at', 'TEXT');
  addCol('payment_completed_at', 'TEXT');
  addCol('payment_failure_reason', 'TEXT');

  exec(`CREATE INDEX IF NOT EXISTS idx_bookings_transaction_id ON bookings(transaction_id)`);
  exec(`CREATE INDEX IF NOT EXISTS idx_bookings_payment_status ON bookings(payment_status)`);

  exec(`
    CREATE TABLE IF NOT EXISTS payment_events (
      id TEXT PRIMARY KEY,
      booking_id TEXT NOT NULL,
      event_type TEXT NOT NULL,
      source TEXT NOT NULL,
      raw_payload TEXT,
      created_at TEXT NOT NULL
    )
  `);
  exec(`CREATE INDEX IF NOT EXISTS idx_payment_events_booking ON payment_events(booking_id)`);
}

// Then call it from the main migrate() function
// migrationAuthorizeNet();
```

Allowed values for payment_status going forward (sql.js doesn't enforce CHECK constraints, but document this in code): - pending — booking created, waiting for customer to complete payment - paid — Authorize.Net confirmed authCapture - failed — declined or otherwise unsuccessful - cancelled — customer clicked Cancel on the hosted page - refunded — admin issued a refund - voided — admin voided before settlement

2.2 Run the migration

```
cd server
node src/migrate.js
```

What to verify: - No errors in the console - sqlite3 bingo.db ".schema bookings" shows the new columns - sqlite3 bingo.db ".schema payment_events" shows the new table

(If you don't have sqlite3 CLI, you can write a tiny Node script: `node -e "const db=require('./src/database.js'); db.getDb().then(()=>console.log(db.all('PRAGMA table_info(bookings)')))"`)

Stage 3 — Environment variables

Goal: Wire up sandbox credentials so the server knows how to talk to Authorize.Net.

Files touched: server/.env, server/.env.example

Add to server/.env:

```
# Authorize.Net — flip ANET_ENV between 'sandbox' and 'production'
ANET_ENV=sandbox
ANET_API_LOGIN_ID=<from project owner>
ANET_TRANSACTION_KEY=<from project owner>
ANET_SIGNATURE_KEY=<from project owner>
ANET_CURRENCY=CAD
```

```
# URLs Authorize.Net will redirect/post to (use your local dev URL during dev)
PUBLIC_BASE_URL=http://localhost:3001
ANET_RETURN_URL=http://localhost:3001/payment/return
ANET_CANCEL_URL=http://localhost:3001/payment/cancel
```

```
# Hold extension during payment (override existing SESSION_HOLD_MINUTES if needed)
SESSION_HOLD_MINUTES=20
```

Add the same keys to server/.env.example but with empty values, so the next dev to clone the repo knows what's needed:

```
ANET_ENV=sandbox
ANET_API_LOGIN_ID=
ANET_TRANSACTION_KEY=
ANET_SIGNATURE_KEY=
ANET_CURRENCY=CAD
PUBLIC_BASE_URL=http://localhost:3001
ANET_RETURN_URL=http://localhost:3001/payment/return
ANET_CANCEL_URL=http://localhost:3001/payment/cancel
SESSION_HOLD_MINUTES=20
```

Make absolutely sure server/.env is in .gitignore. It should already be — verify with `grep -E "\.env|server/.env" .gitignore`.

Stage 4 — Install the SDK

Goal: Add the official Authorize.Net Node SDK.

Files touched: server/package.json

```
cd server
npm install authorizenet
```

What to verify: - server/package.json has "authorizenet": "^1.x.x" in dependencies
- `node -e "const a=require('authorizenet'); console.log(Object.keys(a))"` prints ['APIContracts', 'APIControllers', 'Constants']

Stage 5 — The payments service

Goal: Build a self-contained module that wraps every Authorize.Net API call. The route handlers call this module — they never touch the SDK directly. This makes testing, swapping providers, and reading the code much easier.

Files touched: server/src/services/payments.js (new file)

```
// server/src/services/payments.js
//
// Wrapper around the Authorize.Net SDK.
// Every payment-related external call goes through here.
//
// Functions exposed:
//   createHostedPaymentPage({ bookingId, amount, email, refNumber }) → { token, redirectUrl }
//   verifyTransaction(transId) → { approved, status }
//   verifyWebhookSignature(rawBodyString, signatureHeader) → boolean
//   refundTransaction({ transId, amount, last4 }) → { ok, message }
//   voidTransaction(transId) → { ok, message }

import pkg from 'authorizenet';
const { APIContracts, APIControllers, Constants } = pkg;
import crypto from 'crypto';

const IS_PROD = process.env.ANET_ENV === 'production';
const ENV = IS_PROD ? Constants.endpoint.production : Constants.endpoint.sandbox;

const HOSTED_BASE = IS_PROD
  ? 'https://accept.authorize.net/payment/payment'
  : 'https://test.authorize.net/payment/payment';

function merchantAuth() {
  const m = new APIContracts.MerchantAuthenticationType();
  m.setName(process.env.ANET_API_LOGIN_ID);
  m.setTransactionKey(process.env.ANET_TRANSACTION_KEY);
  return m;
}

// -----
// 1) Create a Hosted Payment Page token
// -----
export function createHostedPaymentPage({ bookingId, amount, email, refNumber }) {
  return new Promise((resolve, reject) => {
    const tx = new APIContracts.TransactionRequestType();
    tx.setTransactionType(APIContracts.TransactionTypeEnum.AUTHCAPTURETRANSACTION);
    tx.setAmount(Number(amount).toFixed(2));
    tx.setCurrencyCode(process.env.ANET_CURRENCY || 'CAD');

    const order = new APIContracts.OrderType();
    order.setInvoiceNumber(refNumber);
    order.setDescription(`Wolastoq Bingo booking ${refNumber}`);
    tx.setOrder(order);

    if (email) {
      const customer = new APIContracts.CustomerDataType();
      customer.setEmail(email);
      tx.setCustomer(customer);
    }

    // Hosted page settings – each setting is a JSON-stringified value
    const settings = [];
    const push = (name, value) => {
      const s = new APIContracts.SettingType();
      s.setSettingName(name);
      s.setSettingValue(JSON.stringify(value));
      settings.push(s);
    };
  });
}
```

```

push('hostedPaymentReturnOptions', {
  showReceipt: false,
  url: `${process.env.ANET_RETURN_URL}?bookingId=${bookingId}`,
  urlText: 'Return to Wolastoq Bingo',
  cancelUrl: `${process.env.ANET_CANCEL_URL}?bookingId=${bookingId}`,
  cancelUrlText: 'Cancel'
});
push('hostedPaymentButtonOptions', { text: 'Pay Now' });
push('hostedPaymentOrderOptions', {
  show: true,
  merchantName: "Saint Mary's Entertainment Centre"
});
push('hostedPaymentPaymentOptions', {
  cardCodeRequired: true,
  showCreditCard: true
});
push('hostedPaymentBillingAddressOptions', { show: true, required: false });
push('hostedPaymentCustomerOptions', {
  showEmail: true,
  requiredEmail: true,
  addPaymentProfile: false
});
push('hostedPaymentSecurityOptions', { captcha: false });
push('hostedPaymentStyleOptions', { bgColor: '#1a365d' }); // optional theme

const arrayList = new APIContracts.ArrayOfSetting();
arrayList.setSetting(settings);

const req = new APIContracts.GetHostedPaymentPageRequest();
req.setMerchantAuthentication(merchantAuth());
req.setTransactionRequest(tx);
req.setHostedPaymentSettings(arrayList);

const ctrl = new APIControllers.GetHostedPaymentPageController(req.getJSON());
ctrl.setEnvironment(ENV);
ctrl.execute() => {
  const apiResponse = ctrl.getResponse();
  const response = new APIContracts.GetHostedPaymentPageResponse(apiResponse);
  if (response && response.getMessages().getResultCode() === APIContracts.MessageTypeE
    resolve({ token: response.getToken(), redirectUrl: HOSTED_BASE });
  } else {
    const msg = response?.getMessages()?.getMessage()?.[0]?.getText() || 'Unknown Auth
    const code = response?.getMessages()?.getMessage()?.[0]?.getCode() || 'UNKNOWN';
    reject(new Error(`[${code}] ${msg}`));
  }
});
});
}

// -----
// 2) Look up a transaction by ID and report whether it was approved
// -----
export function verifyTransaction(transId) {
  return new Promise((resolve, reject) => {
    const req = new APIContracts.GetTransactionDetailsRequest();
    req.setMerchantAuthentication(merchantAuth());
    req.setTransId(String(transId));

    const ctrl = new APIControllers.GetTransactionDetailsController(req.getJSON());
    ctrl.setEnvironment(ENV);

```



```

ctrl.execute(() => {
  const apiResponse = ctrl.getResponse();
  const response = new APIContracts.GetTransactionDetailsResponse(apiResponse);
  if (response && response.getMessages().getResultCode() === APIContracts.MessageTypeEnum.Success) {
    const t = response.getTransaction();
    const responseCode = String(t.getResponseCode());
    resolve({
      approved: responseCode === '1', // 1=approved 2=declined 3=error 4=help
      responseCode,
      status: t.getTransactionStatus(), // e.g. "capturedPendingSettlement"
      authCode: t.getAuthCode(),
      amount: t.getAuthAmount(),
      last4: t.getPayment()?.getCreditCard()?.getCardNumber(),
      settledAt: t.getSubmitTimeUTC?().
    });
  } else {
    const msg = response?.getMessages()?.getMessage()?.[0]?.getText() || 'Lookup failed';
    reject(new Error(msg));
  }
});
});
}

// -----
// 3) Verify webhook signature
// Header format: "X-ANET-Signature: sha512=<HEX>"
// -----
export function verifyWebhookSignature(rawBodyString, signatureHeader) {
  if (!signatureHeader || !process.env.ANET_SIGNATURE_KEY) return false;
  // Authorize.Net signs with HMAC-SHA512 using the Signature Key as bytes
  // The Signature Key is hex – convert to a Buffer
  const keyBuf = Buffer.from(process.env.ANET_SIGNATURE_KEY, 'hex');
  const expectedHex = crypto
    .createHmac('sha512', keyBuf)
    .update(rawBodyString, 'utf8')
    .digest('hex')
    .toUpperCase();
  const provided = signatureHeader.replace(/^sha512=/i, '').toUpperCase();
  if (expectedHex.length !== provided.length) return false;
  return crypto.timingSafeEqual(Buffer.from(expectedHex), Buffer.from(provided));
}

// -----
// 4) Refund (after settlement)
// -----
export function refundTransaction({ transId, amount, last4 }) {
  return new Promise((resolve, reject) => {
    const card = new APIContracts.CreditCardType();
    card.setCardNumber(last4); // Authorize.Net only needs last 4 for refund
    card.setExpirationDate('XXXX');
    const payment = new APIContracts.PaymentType();
    payment.setCreditCard(card);

    const tx = new APIContracts.TransactionRequestType();
    tx.setTransactionType(APIContracts.TransactionTypeEnum.REFUNDTRANSACTION);
    tx.setAmount(Number(amount).toFixed(2));
    tx.setRefTransId(String(transId));
    tx.setPayment(payment);

    const req = new APIContracts.CreateTransactionRequest();

```

```

req.setMerchantAuthentication(merchantAuth());
req.setTransactionRequest(tx);

const ctrl = new APIControllers.CreateTransactionController(req.getJSON());
ctrl.setEnvironment(ENV);
ctrl.execute() => {
  const apiResponse = ctrl.getResponse();
  const response = new APIContracts.CreateTransactionResponse(apiResponse);
  if (response && response.getMessages().getResultCode() === APIContracts.MessageTypeE
    const tr = response.getTransactionResponse();
    if (tr.getResponseCode() === '1') {
      resolve({ ok: true, refundTransId: tr.getTransId() });
    } else {
      const errs = tr.getErrors?.()?.getError?.() || [];
      resolve({ ok: false, message: errs[0]?.getErrorText?.() || 'Refund declined' });
    }
  } else {
    const msg = response?.getMessages()?.getMessage()?.[0]?.getText() || 'Refund failed';
    reject(new Error(msg));
  }
});
});
}

// -----
// 5) Void (before settlement)
// -----
export function voidTransaction(transId) {
  return new Promise((resolve, reject) => {
    const tx = new APIContracts.TransactionRequestType();
    tx.setTransactionType(APIContracts.TransactionTypeEnum.VOIDTRANSACTION);
    tx.setRefTransId(String(transId));

    const req = new APIContracts.CreateTransactionRequest();
    req.setMerchantAuthentication(merchantAuth());
    req.setTransactionRequest(tx);

    const ctrl = new APIControllers.CreateTransactionController(req.getJSON());
    ctrl.setEnvironment(ENV);
    ctrl.execute() => {
      const apiResponse = ctrl.getResponse();
      const response = new APIContracts.CreateTransactionResponse(apiResponse);
      if (response && response.getMessages().getResultCode() === APIContracts.MessageTypeE
        const tr = response.getTransactionResponse();
        if (tr.getResponseCode() === '1') {
          resolve({ ok: true });
        } else {
          resolve({ ok: false, message: 'Void declined – may already be settled' });
        }
      } else {
        const msg = response?.getMessages()?.getMessage()?.[0]?.getText() || 'Void failed';
        reject(new Error(msg));
      }
    }
  });
});
}

```

What to verify: - node -e "import('./src/services/payments.js').then(m => console.log(Object.keys(m)))" lists the five exports - Write a one-off script server/scripts/test-anet.js: js import 'dotenv/config'; import { createHostedPaymentPage } from

```
'../src/services/payments.js';      const r = await createHostedPaymentPage({
bookingId: 'test-1', amount: 25.00, email: 'dev@example.com', refNumber: 'TEST-
1234' }); console.log(r); Run with node server/scripts/test-anet.js. You should get
back{ token: 'long string', redirectUrl: 'https://test.authorize.net/payment/payment'
}. If yes → SDK is wired up correctly.
```

Stage 6 — Refactor the booking endpoint

Goal: Split the existing POST /api/bookings handler into two stages: one that creates a pending booking and asks Authorize.Net for a hosted-page token, and one (later) that confirms the booking after payment.

Files touched: server/src/index.js

6.1 Add helper functions

Find a good spot near the top of index.js (after the existing helpers). Add:

```
// ===== Payment helpers =====
import {
  createHostedPaymentPage,
  verifyTransaction,
  verifyWebhookSignature,
  refundTransaction,
  voidTransaction
} from '../services/payments.js';

function logPaymentEvent(bookingId, eventType, source, payload) {
  run(
    `INSERT INTO payment_events (id, booking_id, event_type, source, raw_payload, created_
    [uuid(), bookingId, eventType, source, JSON.stringify(payload || {}), new Date().toISOString()
    );
  );
}

// Idempotent – safe to call from both /payment/return AND the webhook
function flipBookingToPaid(booking, transId, authCode) {
  // Re-read inside the function to avoid race conditions
  const fresh = get('SELECT * FROM bookings WHERE id = ?', [booking.id]);
  if (!fresh) return { skipped: 'not_found' };
  if (fresh.payment_status === 'paid') return { skipped: 'already_paid' };

  const items = all('SELECT seat_id FROM booking_items WHERE booking_id = ?', [booking.id]);

  run(
    `UPDATE bookings SET payment_status = 'paid', transaction_id = ?, auth_code = ?, payme
    [String(transId || ''), String(authCode || ''), new Date().toISOString(), booking.id]
    );

  for (const it of items) {
    run(`UPDATE seats SET status = 'sold', held_by = NULL, held_until = NULL WHERE id = ?`
  )
}

// Flush to disk immediately – payment-confirming writes are critical
saveDb();

// Emit socket events
for (const it of items) {
  io.to(`session:${booking.session_id}`).emit('seat:sold', { seatId: it.seat_id, session
}
```

```

io.to('admin:receipts').emit('booking:new', { bookingId: booking.id, transId });

// Audit + payment event log
logAudit('booking_paid', 'booking', booking.id, { transId, authCode });
logPaymentEvent(booking.id, 'approved', 'server', { transId, authCode });

// Send confirmation email (don't block the response on this – fire-and-forget)
sendBookingConfirmation(booking.id).catch(err => logger.error('Email send failed', { err }));

return { ok: true };
}

function flipBookingToFailed(booking, reason) {
  const fresh = get('SELECT * FROM bookings WHERE id = ?', [booking.id]);
  if (!fresh || fresh.payment_status === 'paid') return;
  run(
    `UPDATE bookings SET payment_status = 'failed', payment_failure_reason = ? WHERE id = ?`
    [String(reason).slice(0, 500), booking.id]
  );
  // Leave seats as 'held' – the holds.js cleanup will release them on expiry.
  saveDb();
  logPaymentEvent(booking.id, 'failed', 'server', { reason });
}

```

6.2 Refactor POST /api/bookings → POST /api/bookings/initiate

Find the existing `app.post('/api/bookings', ...)` block (around line 233 in current code).

Keep all the validation logic (session lookup, seat-hold check, PHD inventory check, attendee validation, totalAmount calculation). **Change** the parts after totalAmount is computed:

```

// REPLACE the existing app.post('/api/bookings', ...) handler with this:
app.post('/api/bookings/initiate', async (req, res) => {
  const { sessionId, holderId, attendees, email } = req.body;

  // ... [keep all your existing validation: required fields, email regex,
  //      session lookup, seat-hold check, package lookup, PHD inventory check] ...

  try {
    let totalAmount = 0;
    const bookingId = uuid();
    const refNumber = generateRef();

    // Insert booking_items + booking_addons EXACTLY like before
    for (const att of attendees) {
      const itemId = uuid();
      const itemRef = generateRef();
      totalAmount += requiredPkg.price;

      run('INSERT INTO booking_items (id, booking_id, first_name, last_name, seat_id, pack
        [itemId, bookingId, att.firstName, att.lastName, att.seatId, requiredPkg.id, requi

      // [!] DO NOT update seats to 'sold' here – that happens after payment confirms
      // Instead, EXTEND the hold so it doesn't expire mid-payment
      const newHoldUntil = new Date(Date.now() + HOLD_MINUTES * 60 * 1000).toISOString();
      run('UPDATE seats SET held_until = ? WHERE id = ?', [newHoldUntil, att.seatId]);

      if (att.addons) {
        for (const addon of att.addons) {
          const pkg = useSessionPkgs

```

```

        ? sessionPkgs.find(p => p.id === addon.packageId)
        : get('SELECT * FROM packages WHERE id = ? AND is_active = 1', [addon.packageId])
    if (pkg) {
        const addonPrice = pkg.price * addon.quantity;
        totalAmount += addonPrice;
        run('INSERT INTO booking_addons (id, booking_item_id, package_id, quantity, price) VALUES (
            [uuid(), itemId, addon.packageId, addon.quantity, addonPrice]);
    }
}
}

// [!] Insert booking as 'pending' – NOT 'paid'
run(
    'INSERT INTO bookings (id, session_id, reference_number, total_amount, payment_status) VALUES (
        [bookingId, sessionId, refNumber, totalAmount, 'pending', new Date().toISOString(),
    );
saveDb();
logPaymentEvent(bookingId, 'created_pending', 'server', { totalAmount });

// Now ask Authorize.Net for a hosted-page token
const { token, redirectUrl } = await createHostedPaymentPage({
    bookingId,
    amount: totalAmount,
    email: trimmedEmail,
    refNumber
});

run(
    'UPDATE bookings SET hosted_token = ?, payment_attempted_at = ? WHERE id = ?',
    [token, new Date().toISOString(), bookingId]
);
saveDb();
logPaymentEvent(bookingId, 'hosted_token_issued', 'server', { tokenPreview: token.slice(0, 20) });

return res.json({ bookingId, redirectUrl, token });
} catch (err) {
    logger.error('Booking initiate failed', { err: err.message });
    return res.status(502).json({ error: 'Could not start payment. Please try again.' });
}
});

```

6.3 Add the return / cancel / status / webhook endpoints

```

// ===== /payment/return =====
// Authorize.Net redirects the customer's browser here after payment.
// We don't trust this redirect alone (could be tampered with) – we redirect
// to a "processing" page that polls /status. The webhook actually flips the booking.
app.get('/payment/return', (req, res) => {
    const { bookingId } = req.query;
    if (!bookingId) return res.redirect('/');
    return res.redirect(`/booking/${bookingId}/processing`);
});

app.post('/payment/return', express.urlencoded({ extended: true }), (req, res) => {
    // Authorize.Net may also POST here with the transaction details as form fields
    const bookingId = req.query.bookingId || req.body?.bookingId;
    if (!bookingId) return res.redirect('/');
    return res.redirect(`/booking/${bookingId}/processing`);
});

```

```

});

// ===== /payment/cancel =====
app.get('/payment/cancel', (req, res) => {
  const { bookingId } = req.query;
  if (bookingId) {
    const b = get('SELECT * FROM bookings WHERE id = ?', [bookingId]);
    if (b && b.payment_status === 'pending') {
      run('UPDATE bookings SET payment_status = 'cancelled' WHERE id = ?', [bookingId]);
      saveDb();
      logPaymentEvent(bookingId, 'cancelled_by_user', 'server', {});
    }
  }
  return res.redirect(`/booking/${bookingId || ''}/cancelled`);
});

// ===== /api/bookings/:id/status =====
// The frontend "processing" page polls this every 2s
app.get('/api/bookings/:id/status', (req, res) => {
  const b = get('SELECT id, payment_status, transaction_id, payment_failure_reason FROM bookings WHERE id = ?', [req.params.id]);
  if (!b) return res.status(404).json({ error: 'Not found' });
  res.json(b);
});

// ===== /api/webhooks/authorize-net =====
// Authorize.Net pings this endpoint when transactions change state.
// We use it as the SOURCE OF TRUTH – the customer-facing /payment/return is
// just for UX redirection.
app.post(
  '/api/webhooks/authorize-net',
  express.raw({ type: 'application/json' }), // raw body needed for HMAC
  async (req, res) => {
    const raw = req.body.toString('utf8');
    const sig = req.get('X-ANET-Signature');

    if (!verifyWebhookSignature(raw, sig)) {
      logger.warn('Webhook signature invalid', { sig });
      return res.status(401).end();
    }

    let event;
    try { event = JSON.parse(raw); }
    catch { return res.status(400).end(); }

    const { eventType, payload } = event;
    const transId = payload?.id;
    const invoiceNumber = payload?.invoiceNumber || payload?.merchantReferenceId;

    const booking = invoiceNumber
      ? get('SELECT * FROM bookings WHERE reference_number = ?', [invoiceNumber])
      : null;

    if (!booking) {
      // ack so Authorize.Net stops retrying – we just don't have a matching booking
      return res.status(202).end();
    }

    logPaymentEvent(booking.id, 'webhook_received', 'authorize_net', { eventType, transId });

    try {

```

```

    if (eventType === 'net.authorize.payment.authcapture.created') {
      const v = await verifyTransaction(transId);
      if (v.approved) flipBookingToPaid(booking, transId, v.authCode);
      else flipBookingToFailed(booking, `responseCode=${v.responseCode}`);
    } else if (eventType === 'net.authorize.payment.refund.created') {
      run(`UPDATE bookings SET payment_status = 'refunded' WHERE id = ?`, [booking.id]);
      saveDb();
      logPaymentEvent(booking.id, 'refunded', 'authorize_net', { transId });
    } else if (eventType === 'net.authorize.payment.void.created') {
      run(`UPDATE bookings SET payment_status = 'voided' WHERE id = ?`, [booking.id]);
      saveDb();
      logPaymentEvent(booking.id, 'voided', 'authorize_net', { transId });
    }
  } catch (err) {
    logger.error('Webhook handler error', { err: err.message, eventType });
  }

  return res.status(200).end();
}
);

```

6.4 What to do with the OLD POST /api/bookings

Don't delete it — keep it gated behind admin auth so staff can create comp/no-payment bookings:

```

// Old endpoint becomes admin-only
app.post('/api/bookings', adminAuth, (req, res) => {
  // ... existing handler unchanged, but now only admins can hit it ...
});

```

What to verify after Stage 6: - Server starts without errors: `npm run dev:server` - POST to `/api/bookings/initiate` from your existing booking UI returns `{ bookingId, redirectUrl, token }` (you can test with curl using a valid sessionId/holderId/attendees) - Sandbox dashboard shows the request in the API logs (Account → Settings → API Logs)

Stage 7 — Frontend changes

Goal: Switch the booking submit to the new flow, add a “processing” page, add a “cancelled” page.

Files touched: `client/src/api.js`, plus 2-3 new component files.

7.1 Update the booking submit

Find where the booking is currently posted in `client/src/`. Replace the call:

```

// client/src/api.js – replace your existing createBooking() with this
export async function createBooking(payload) {
  const res = await fetch('/api/bookings/initiate', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload)
  });
  if (!res.ok) {
    const { error } = await res.json().catch(() => ({}));
    throw new Error(error || 'Could not start payment');
  }
  const { bookingId, redirectUrl, token } = await res.json();
}

```

```

// Auto-submit a hidden form to the Authorize.Net hosted page
const form = document.createElement('form');
form.method = 'POST';
form.action = redirectUrl;
form.style.display = 'none';
const input = document.createElement('input');
input.type = 'hidden';
input.name = 'token';
input.value = token;
form.appendChild(input);
document.body.appendChild(form);
form.submit();
// Browser navigates away – anything below this line never runs
}

```

7.2 Processing page

Create client/src/pages/BookingProcessing.jsx:

```

import { useEffect, useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';

export default function BookingProcessing() {
  const { id } = useParams();
  const navigate = useNavigate();
  const [status, setStatus] = useState('pending');
  const [tries, setTries] = useState(0);

  useEffect(() => {
    if (status === 'paid') {
      navigate(`/booking/${id}`, { replace: true });
      return;
    }
    if (status === 'failed' || status === 'cancelled') return;

    const t = setTimeout(async () => {
      try {
        const res = await fetch(`/api/bookings/${id}/status`);
        const data = await res.json();
        setStatus(data.payment_status);
        setTries(n => n + 1);
      } catch { setTries(n => n + 1); }
    }, 2000);
    return () => clearTimeout(t);
  }, [status, tries, id, navigate]);

  if (status === 'failed') {
    return (
      <div className="max-w-md mx-auto p-8 text-center">
        <h1 className="text-2xl font-bold text-red-600">Payment was not completed</h1>
        <p className="my-4">Your card was declined or the payment did not go through.</p>
        <a href={`/${id}`} className="px-4 py-2 bg-blue-600 text-white rounded">Try again</a>
      </div>
    );
  }

  if (status === 'cancelled') {
    return (
      <div className="max-w-md mx-auto p-8 text-center">
        <h1 className="text-2xl font-bold">Payment cancelled</h1>
        <p>You can return to the home page and try again any time.</p>
      </div>
    );
  }
}

```



```

    </div>
  );
}
return (
  <div className="max-w-md mx-auto p-8 text-center">
    <div className="animate-spin h-12 w-12 border-4 border-blue-600 border-t-transparent">
      <h1 className="text-xl font-semibold">Confirming your payment...</h1>
      <p className="mt-2 text-gray-600">This usually takes a few seconds. Please don't close the browser.</p>
      {tries > 30 && (
        <p className="mt-4 text-sm text-gray-500">Still working – if your payment was successful, please try again.</p>
      )}
    </div>
  );
}

```

7.3 Wire up the routes

In wherever your React Router routes are declared:

```

import BookingProcessing from './pages/BookingProcessing';

<Route path="/booking/:id/processing" element={<BookingProcessing />} />
<Route path="/booking/:id/cancelled" element={<BookingCancelled />} /> /* simple "cancelled" route */
/* The existing /booking/:id receipt route stays as-is */

```

7.4 Build the client

```
npm run build
```

This is required because server/src/index.js serves ../client/dist. The dev server proxy will work without it for local dev, but the production server needs the build.

What to verify after Stage 7: - Run the app, place a booking - Browser redirects to <https://test.authorize.net/payment/payment> and shows the Authorize.Net card form - Use sandbox card 4111 1111 1111 1111, exp 12/30, CVV 123 → form returns to your processing page → polls until paid → redirects to receipt

[INSERT SCREENSHOT HERE] — Authorize.Net hosted page showing the card form, with the “Pay Now” button at the bottom. The page URL will start with <https://test.authorize.net/payment/payment>

Stage 8 — Webhook setup

Goal: Tell Authorize.Net to ping your server on payment events.

Files touched: none (this is configuration on Authorize.Net’s side).

8.1 Expose your local server to the internet

For local testing, install ngrok (or use Cloudflare Tunnel):

```

# In a new terminal, while server is running on :3001
ngrok http 3001

```

Note the HTTPS URL ngrok gives you, e.g. <https://abc123.ngrok-free.app>.

8.2 Register the webhook

In the sandbox dashboard:

1. **Account → Settings → Webhooks** → click “+ Add Endpoint”
2. URL: `https://abc123.ngrok-free.app/api/webhooks/authorize-net` (or your real domain in production)
3. Select these events:
 - `net.authorize.payment.authcapture.created`
 - `net.authorize.payment.refund.created`
 - `net.authorize.payment.void.created`
 - (optional) `net.authorize.payment.fraud.held`
4. Status: Active
5. Save

[INSERT SCREENSHOT HERE] — Authorize.Net dashboard, Webhooks page, showing the new endpoint with green “Active” status and the list of subscribed events.

8.3 Test the webhook

In the dashboard webhook list, click your endpoint → “**Send Test Notification**”. Choose `authcapture.created`. Click Send.

What to verify: - Your server console shows the webhook hit (you can `console.log` or check `payment_events` table) - Status code returned was 200 (visible in the dashboard’s recent deliveries list) - If signature verification failed (401), the `ANET_SIGNATURE_KEY` env var is wrong

[INSERT SCREENSHOT HERE] — Webhook detail page showing recent deliveries, all with green “200” status codes.

Stage 9 — Admin refund/void UI

Goal: Let staff issue refunds or voids from the admin dashboard.

Files touched: `server/src/index.js` (new endpoints), wherever your admin booking detail UI lives.

9.1 Backend endpoints

```
// server/src/index.js

// Void – only works before settlement
app.post('/api/admin/bookings/:id/void', adminAuth, async (req, res) => {
  const b = get('SELECT * FROM bookings WHERE id = ?', [req.params.id]);
  if (!b) return res.status(404).json({ error: 'Not found' });
  if (b.payment_status !== 'paid') return res.status(400).json({ error: 'Booking is not paid' });
  if (!b.transaction_id) return res.status(400).json({ error: 'No transaction to void' });

  const result = await voidTransaction(b.transaction_id);
  if (!result.ok) return res.status(400).json({ error: result.message });

  run(`UPDATE bookings SET payment_status = 'voided' WHERE id = ?`, [b.id]);
  // Optional: release seats back to vacant if the session hasn't happened yet
  const items = all('SELECT seat_id FROM booking_items WHERE booking_id = ?', [b.id]);
  for (const it of items) {
    run(`UPDATE seats SET status = 'vacant', held_by = NULL, held_until = NULL WHERE id = ?`, [it.seat_id]);
  }
  saveDb();
  logAudit('booking_voided', 'booking', b.id, { admin: req.user?.username });
  logPaymentEvent(b.id, 'voided', 'admin', { admin: req.user?.username });
  res.json({ ok: true });
});
```

```
// Refund – works after settlement
app.post('/api/admin/bookings/:id/refund', adminAuth, async (req, res) => {
  const b = get('SELECT * FROM bookings WHERE id = ?', [req.params.id]);
  if (!b) return res.status(404).json({ error: 'Not found' });
  if (b.payment_status !== 'paid') return res.status(400).json({ error: 'Booking is not paid' });
  if (!b.transaction_id) return res.status(400).json({ error: 'No transaction to refund' });

  // Need last 4 – pull it from Authorize.Net
  const v = await verifyTransaction(b.transaction_id);
  const last4 = (v.last4 || '').replace(/[^\d]/g, '').slice(-4);
  if (!last4) return res.status(500).json({ error: 'Could not retrieve card last 4' });

  const result = await refundTransaction({
    transId: b.transaction_id,
    amount: req.body?.amount || b.total_amount,
    last4
  });
  if (!result.ok) return res.status(400).json({ error: result.message || 'Refund declined' });

  run(`UPDATE bookings SET payment_status = 'refunded' WHERE id = ?`, [b.id]);
  saveDb();
  logAudit('booking_refunded', 'booking', b.id, { admin: req.user?.username });
  logPaymentEvent(b.id, 'refunded', 'admin', { admin: req.user?.username, amount: req.body?.amount });
  res.json({ ok: true, refundTransId: result.refundTransId });
});
```

9.2 Admin UI

Add a “Refund” / “Void” button to the existing admin booking detail view. Pseudo-code:

```
{booking.payment_status === 'paid' && (
  <>
    <button onClick={() => onVoid(booking.id)}>Void (before settlement)</button>
    <button onClick={() => onRefund(booking.id)}>Refund</button>
  </>
)}
{booking.payment_status === 'refunded' && <span className="text-green-600">Refunded</span>}
{booking.payment_status === 'voided' && <span className="text-gray-600">Voided</span>}
```

The action handlers POST to /api/admin/bookings/:id/void or /refund and reload the booking on success. Authorize.Net automatically tries void first if you call refund on an unsettled transaction — but the Node SDK doesn’t, so we expose two buttons.

Stage 10 — End-to-end testing

Walk through every case before you call it done. Use these sandbox cards:

Card number	Result
4111 1111 1111 1111	Approved (Visa)
5424 0000 0000 0015	Approved (Mastercard)
4222 2222 2222 2	Decline
4007 0000 0002 7	Decline – insufficient funds
Any past expiration	Approved (sandbox is forgiving)
CVV: 900	Triggers CVV mismatch decline

Test cases:

- ❑ **Happy path:** held seats → initiate → pay → return → seats sold, email arrives
 - ❑ **Tab close after payment:** start payment, close the browser tab right after entering card. Within ~30 seconds the webhook should flip the booking to paid anyway.
 - ❑ **Cancel button on hosted page:** clicks Cancel → returns to /payment/cancel → booking is cancelled, seats stay held until they expire naturally.
 - ❑ **Decline:** use 4222 2222 2222 2 → returns to processing page → status flips to failed → user sees retry button.
 - ❑ **Hold expires during payment:** set SESSION_HOLD_MINUTES=1 temporarily, start a booking, wait 90 seconds, finish payment. The webhook will still flip it to paid but the seats will already be vacant — handle this gracefully (you can re-acquire the seats if still vacant, or refund automatically).
 - ❑ **Concurrent bookings on the same seat:** two browsers race for the same seat. Only one should be able to initiate. The second should fail at the seat-hold validation step with 409.
 - ❑ **Webhook fires twice:** in dashboard, click “Replay” on a delivered webhook. Booking should remain paid, no duplicate emails sent.
 - ❑ **Webhook with bad signature:** manually change one byte of ANET_SIGNATURE_KEY in .env, restart server, send a test webhook. Server should return 401.
 - ❑ **Refund:** book → pay → wait until settled (sandbox auto-settles within a few hours, or use the dashboard “Force Settle” button) → admin clicks Refund → booking shows refunded.
 - ❑ **Void before settlement:** book → pay → admin clicks Void within minutes → booking shows voided, seats go back to vacant.
 - ❑ **Total amount integrity:** book a multi-attendee, multi-package order. Confirm the amount sent to Authorize.Net matches your total_amount in the database EXACTLY (including PHD add-ons).
 - ❑ **PHD inventory:** book with PHD add-ons → after payment, /api/phd-inventory returns lower remaining count.
-

Stage 11 — Production cutover

When all sandbox tests pass and the merchant account is live in production:

1. Get production API Login ID, Transaction Key, Signature Key from the project owner.
 2. Update production env vars on your real host (NOT in .env — use whatever secret manager your host provides):


```
ANET_ENV=production
ANET_API_LOGIN_ID=<production>
ANET_TRANSACTION_KEY=<production>
ANET_SIGNATURE_KEY=<production>
PUBLIC_BASE_URL=https://your-real-domain.example
ANET_RETURN_URL=https://your-real-domain.example/payment/return
ANET_CANCEL_URL=https://your-real-domain.example/payment/cancel
```
 3. In the **production** Authorize.Net dashboard (login at <https://account.authorize.net>), repeat Stage 8.2 to register the webhook for your real domain.
 4. Deploy.
 5. **Smoke test with a real card for \$1.** Refund it immediately. Confirm:
 - Booking ends up paid with a real transaction_id
 - The transaction appears in the production Authorize.Net dashboard
 - The refund clears
 - Confirmation email reached the customer
 6. Tell support staff what to do if a customer reports a stuck booking (look it up by reference number, check payment_status, check payment_events for the trail).
-

Appendix A — Common errors and what they mean

Error	Cause	Fix
E00007 User authentication failed	Wrong API Login ID or Transaction Key	Re-check .env against dashboard
E00027 Transaction was unsuccessful	Card declined	Show “card declined” UX, let user retry
E00114 Invalid OTS Token	Token expired (15 min limit) or already used	Refresh token by re-initiating
E00003 Element ‘currencyCode’ is invalid 401 from your own webhook	Currency not allowed for this merchant Signature key mismatch	Confirm merchant supports CAD Make sure ANET_SIGNATURE_KEY in env matches dashboard exactly
Webhook never fires	Endpoint URL not reachable from internet	ngrok in dev / firewall/DNS in prod

Appendix B — Files modified by this integration

server/	
├─ package.json	(+ "authorizenet" dep)
├─ .env	(+ ANET_* vars)
├─ .env.example	(+ ANET_* vars)
└─ src/	
├─ index.js	(refactored bookings, +4 routes)
├─ migrate.js	(+ migrationAuthorizeNet)
└─ services/	
└─ payments.js	(NEW – wrapper around Authorize.Net SDK)
client/	
└─ src/	
├─ api.js	(createBooking refactored)
└─ pages/	
├─ BookingProcessing.jsx	(NEW – polls /status)
└─ BookingCancelled.jsx	(NEW – simple message page)

Last updated: 2026-05-07. Questions? Read `AUTHORIZE-NET-INTEGRATION.md` in the same folder for the high-level architecture rationale behind these choices.